



OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Communication Patterns

Archana A

Department of Computer Applications

Communication Pattern

Most of the today's software applications run on distributed systems. These **distributed systems need a means for communication.**

Problems:

1. There are **many communication mechanisms** to choose.
 - The **use of communication facilities** is often hard-wired into existing applications, leading to various problems.
 - **Difficult to change** the communication mechanism later.
2. **Portability** - Migration of sub systems from one network node to another is only possible if the communication facility allows it

Solution:

- Loosen the coupling between components of a distributed system and the mechanism it uses for communication

eg: by using **Encapsulation** and **Location transparency**

Important **aspects of communication patterns** are:

Encapsulation of communication facilities.

1. Hiding the details of the underlying communication mechanism from the users.
2. Abstract programming interface is provided on top of the low-level communication facilities.

Location Transparency

1. Applications are allowed to access remote components without any knowledge of their physical location.

Name: Publisher-Subscriber Pattern

- The Publisher-subscriber design pattern helps to keep the state of **cooperating components synchronized**.
- To achieve this it enables **one-way propagation of changes**; one publisher notifies any number of subscribers about the changes to its state.

Also Known as:

Observers, Dependents

Problem:

A situation often arises in which data changes in one place, but many other components depend on this data.

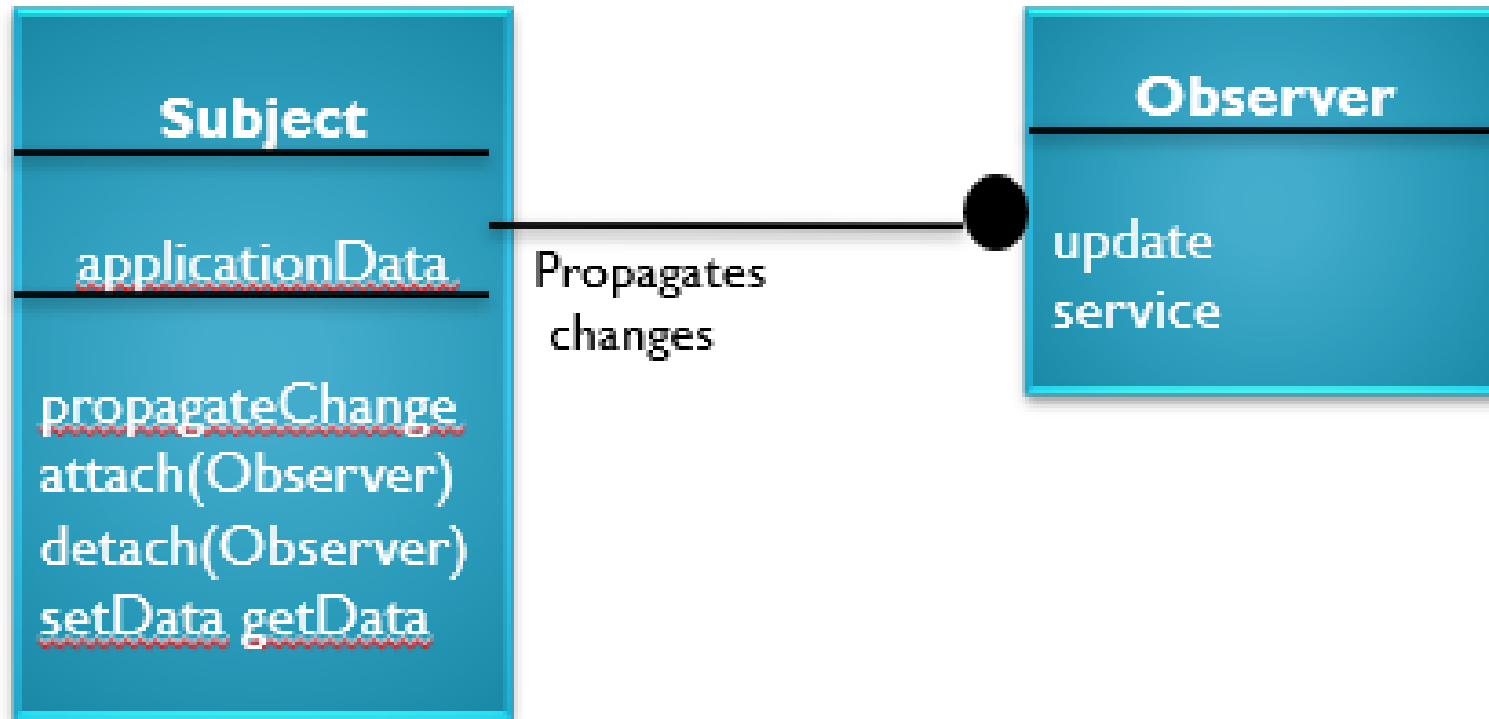
- The solution should balance the following forces:
 - One or more components must be **notified about changes** in a particular component .
 - This number & identities of dependent components is not known in prior or may even change over time.
 - Explicit polling by dependents for new information is not feasible.
 - The information publisher and its dependents should not be tightly coupled when introducing a change-propagation mechanism.

Solution:

- The ***subject*** — Implement a change propagation(spreading) mechanism between the information provider
- the ***observers***—the components dependent on it
- Observers can dynamically register or unregister with this mechanism.
- Whenever the subject changes its state, it starts the change-propagation mechanism to restore consistency with all registered observers.
- Changes are propagated by invoking a special update

OBJECT ORIENTED SOFTWARE ENGINEERING

Patterns - structure



Variants:

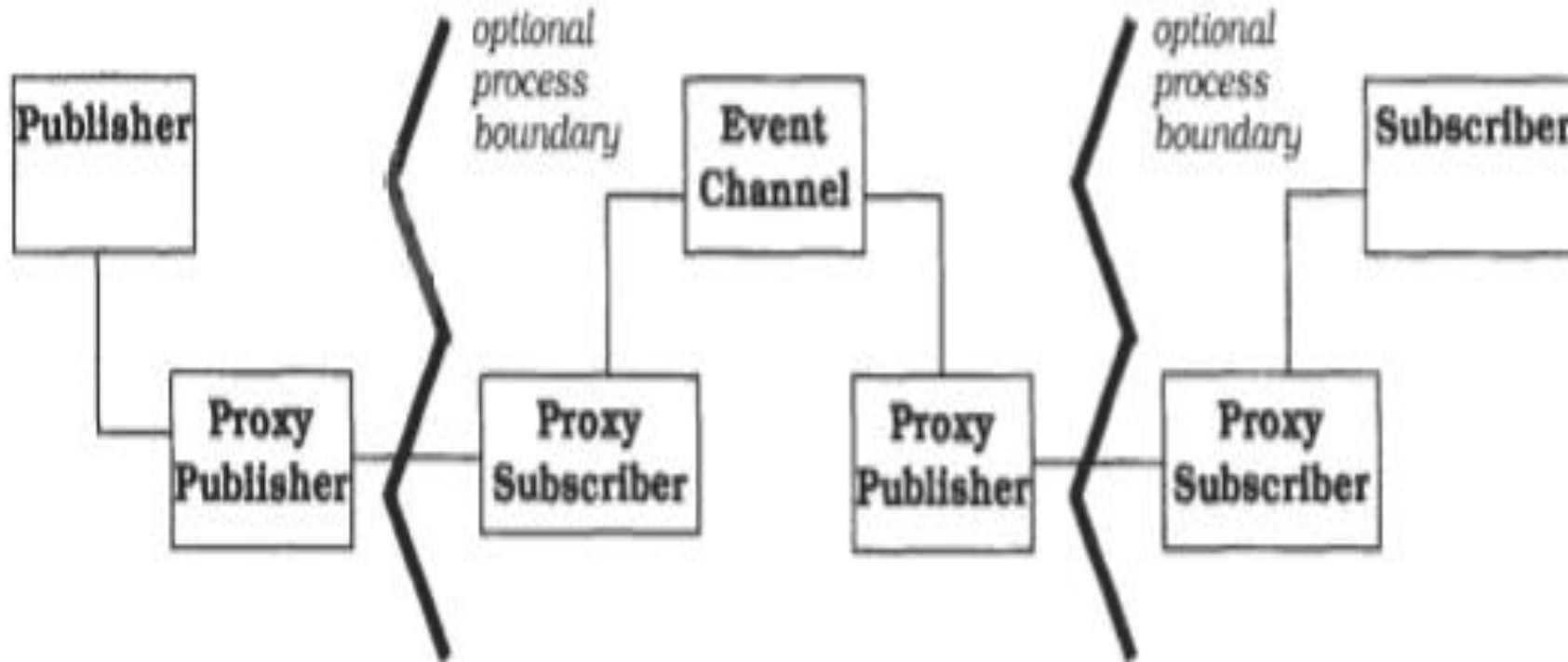
GateKeeper:

- Publisher-Subscriber pattern can also be applied **to distributed systems**. In this variant of publisher instance in **one process notifies remote subscribers**.
- The publisher may alternatively be spread over two processes. In one process a component sends out messages, while in the receiving process a singleton 'gatekeeper' de-multiplexes them by surveying the entry points to the process.
- The gatekeeper notifies event-handling subscribers when events for which they registered occur.

OBJECT ORIENTED SOFTWARE ENGINEERING

Publisher-Subscriber Pattern

General Structure:



Social Media Feed Updates:

Publishers: Users creating posts, commenting, liking/reacting

Messages: New posts, comments, likes/reactions

Subscribers: User feeds, notification systems

Auction Bidding Updates:

- Publishers:** Auction platform, bidders placing bids

- Messages:** New bids, auction status changes (started, ended)

- Subscribers:** Other bidders, auction observers

Internet of Things (IoT) Sensor Data:

Publishers: IoT devices (temperature sensors, motion detectors)

Messages: Sensor readings, device status updates

Subscribers: Monitoring applications, data analysis tools, automation systems



THANK YOU

Ms. Archana A

Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392